

DESIGN DAN IMPLEMENTASI SISTEM NAVIGASI MAZE ROBOT MOBILE PIONEER P3DX BERBASIS COGNITIVE LAYER LARGE LANGUAGE MODEL (LLM) DAN EXECUTION LAYER CLOSED-LOOP PID CONTROLLER

Author(s): Aisyah Ayu Fitriani
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Robot Otonom
YouTube: <https://youtu.be/pD1YHgxE-d-Q>
GitHub: https://github.com/Asyah24/FP_SR0

PENJELASAN STRUKTUR KODINGAN

Kodingan ini menerapkan arsitektur *hybrid control system* (sistem kendali hibrida) pada robot mobile PioneerP3DX di simulator CoppeliaSim. Sistem ini menggabungkan dua metode utama:

- **AI High-Level Decision Making (Cognitive Layer):** Menggunakan Large Language Model (LLM) lokal via Ollama (Qwen2.5:1.5b) untuk menganalisis kondisi lingkungan (*state*) dan menentukan strategi makro (*body space action*).
- **Low-Level Feedback Control (Execution Layer):** Menggunakan kendali *closed-loop* PID untuk menerjemahkan perintah makro menjadi nilai kecepatan motor roda kiri dan kanan secara riil (*joint space speed*).

Pembagian Blok Sistem

1. **Blok 1 (Konfigurasi & Sensor):** Kelas `Config` menyimpan seluruh parameter fisik robot, ambang batas deteksi sensor, parameter tuning PID, dan alamat endpoint API Ollama. Kelas `SensorProcessor` berfungsi menyederhanakan data mentah 16 sensor sonar PioneerP3DX menjadi 6 arah utama menggunakan fungsi `min()` untuk mengambil jarak terdekat dari objek/halangan.
2. **Blok 2 (Kendali PID):** Kelas `PIDController` menghitung nilai koreksi berdasarkan *error* (selisih jarak aktual robot ke dinding kanan terhadap jarak ideal `wall_dist`). Blok ini telah diperbaiki dari masalah windup integral dengan membatasi akumulasi nilai melalui `integral_limit`.
3. **Blok 3 (Interface AI):** Fungsi `ai_agent_interface(state)` bertindak sebagai jembatan *state-action*. Fungsi ini menyusun *prompt* terstruktur berformat JSON yang berisi koordinat posisi, jarak ke goal, dan data kedekatan sensor. Jika server LLM mengalami *stuttering* atau *timeout*, sistem *fallback* berbasis aturan (*rule-based*) akan langsung mengambil alih kendali demi keamanan robot.
4. **Blok 4 (Main Loop & Translasi Ruang):** Fungsi `main()` menggerakkan simulasi. Untuk mencegah kelambatan (*lag*) akibat waktu tunggu (*latency*) pemrosesan bahasa, pemanggilan LLM dijadwalkan hanya sekali setiap 6 loop. Di dalam blok ini terjadi proses konversi dari perintah taktis bahasa (*body space*) ke sinyal kinematika roda (*joint space*).

1 ABSTRAK

Penelitian ini membahas mengenai perancangan sistem kendali navigasi otonom pada robot mobile Pioneer3DX di dalam lingkungan labirin (*maze*) menggunakan simulator CoppeliaSim. Arsitektur kendali yang diajukan memisahkan proses pengambilan keputusan tingkat tinggi (*high-level decision making*) dan eksekusi gerak tingkat rendah (*low-level execution*). *Cognitive layer* diimplementasikan menggunakan Large Language Model (LLM) lokal Qwen2.5:1.5b melalui framework Ollama untuk menentukan aksi makro berdasarkan representasi dataset *state* robot. Sementara itu, *execution layer* menggunakan kendali *closed-loop* Proportional-Integral-Derivative (PID) untuk menjaga stabilitas gerak menyisir dinding (*wall following*). Hasil pengujian menunjukkan bahwa integrasi kedua metode ini mampu menavigasikan robot menuju koordinat target tanpa mengalami kolisi dengan memanfaatkan sistem *fallback* otomatis saat terjadi *latency* pada jaringan API.

2 PENDAHULUAN

2.1 Latar Belakang

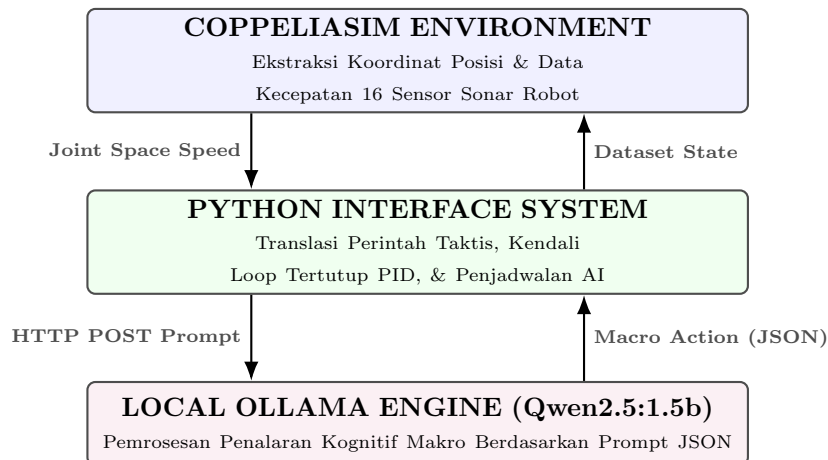
Navigasi robot otonom di dalam lingkungan labirin yang dinamis memerlukan sistem yang tidak hanya adaptif terhadap perubahan halangan, melainkan juga presisi dalam kalkulasi gerak fisik. Pendekatan konvensional seperti *rule-based system* sering kali kaku dan membutuhkan algoritma logika yang kompleks jika kondisi lingkungan berkembang. Dengan perkembangan Artificial Intelligence (AI), Large Language Model (LLM) tingkat ringan kini dapat dieksekusi secara lokal untuk melakukan penalaran kontekstual terhadap data sensorik robot (*state*). Namun, LLM tidak dapat menggerakkan aktuator secara halus secara langsung. Oleh karena itu, diperlukan sebuah sistem interface yang mampu menerjemahkan keputusan semantik dari AI (*body space*) menjadi parameter kontrol kinematika roda (*joint space*) berbasis kontroler klasik PID.

2.2 Tujuan

- Merancang arsitektur jembatan komunikasi (*interface*) data antara lingkungan simulator CoppeliaSim dengan LLM lokal.
- Mengimplementasikan kontroler PID *closed-loop* yang bebas dari masalah *integral windup* untuk kestabilan manuver robot.
- Menganalisis performa navigasi robot hibrida dalam mencapai titik koordinat *goal* yang telah ditentukan.

3 METODOLOGI DAN ARSITEKTUR SISTEM

Sistem kendali navigasi ini dibagi menjadi tiga lapisan arsitektur utama yang bekerja bersama secara siklikal. Alur pertukaran informasi data interaktif tersebut ditunjukkan secara ringkas melalui diagram blok di bawah ini:



3.1 Pemrosesan Data State (State Extraction)

Data posisi robot 3D (X, Y, Z) diperoleh secara langsung dari *object handle* di CoppeliaSim. Jarak Euclidean terhadap titik target dihitung menggunakan persamaan:

$$d_{\text{goal}} = \sqrt{(X_{\text{goal}} - X_{\text{robot}})^2 + (Y_{\text{goal}} - Y_{\text{robot}})^2}$$

Data sensor ultrasonik sebanyak 16 channel direduksi menjadi 6 kluster arah strategis melalui operasi matematika nilai minimum untuk meminimalkan beban komputasi pemrosesan string *prompt* pada LLM.

3.2 Translasi Ruang Kerja (Body-to-Joint Space Translation)

Keputusan makro dari AI (*body space action*) ditranslasikan menjadi parameter fungsional roda (*joint space speed*) sebagai berikut:

- **STOP_REACHED_GOAL / REACHED_GOAL:** Menghentikan perputaran kedua roda secara instan ($V_L = 0, V_R = 0$) ketika jarak robot ke target $d_{\text{goal}} < 0.3$ m.
- **TURN_LEFT_90:** Menggunakan pendekatan *open-loop preset* di mana roda kiri dan kanan diberikan nilai inversi konstan ($V_L = -0.5 \times V_{\text{base}}, V_R = 0.5 \times V_{\text{base}}$) untuk memutar badan robot menjauhi dinding depan.
- **FOLLOW_WALL_RIGHT:** Menggunakan pendekatan *closed-loop feedback* berbasis kendali PID. Sinyal koreksi tikungan ($u(t)$) dihitung berdasarkan deviasi jarak dinding kanan terhadap batas ideal:

$$e(t) = \text{Sonar}_{\text{front_right}} - d_{\text{wall}}$$

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Kecepatan akhir pada motor joint diatur secara silang (*cross-coupling*) untuk mencegah benturan pada sisi berlawanan:

$$V_L = V_{\text{base}} - u(t)$$

$$V_R = V_{\text{base}} + u(t)$$

4 IMPLEMENTASI KODE PROGRAM

Sistem diimplementasikan menggunakan bahasa pemrograman Python 3 dengan pustaka eksternal `requests` untuk melakukan komunikasi data *stateless* HTTP ke server lokal Ollama pada port 11434. Struktur kendali utama dijalankan di dalam looping `while` dengan interval jeda aman `time.sleep(0.04)` untuk menjaga siklus sinkronisasi frekuensi simulator.

```
1 from enum import IntEnum
2 from dataclasses import dataclass, asdict
3 import time
4 import json
5 import requests
6 import coppelia_client
7 import signal
8
9 @dataclass
10 class Config:
11     base_speed: float = 0.55
12     wall_dist: float = 0.25
13     front_turn_dist: float = 0.45
14     emergency_dist: float = 0.13
15
16     kp: float = 14.0
17     ki: float = 0.1
18     kd: float = 14.0
19     avoid_gain: float = 5.0
20     max_turn: float = 0.8
21
22     ollama_url: str = "http://localhost:11434/api/generate"
23     model_name: str = "qwen2.5:1.5b"
24     debug: bool = True
25
26 GOAL_POSITION = [2.5, -4.0]
```

```

27
28 @dataclass
29 class SensorData:
30     front_left: float; front_center: float; front_right: float
31     back_left: float; back_center: float; back_right: float
32
33 class SensorProcessor:
34     @staticmethod
35     def process(readings: list) -> SensorData:
36         return SensorData(
37             front_left=min(readings[0], readings[1], readings[2]),
38             front_center=min(readings[3], readings[4]),
39             front_right=min(readings[5], readings[6], readings[7]),
40             back_left=min(readings[13], readings[14], readings[15]),
41             back_center=min(readings[11], readings[12]),
42             back_right=min(readings[8], readings[9], readings[10])
43         )
44
45 class PIDController:
46     def __init__(self, kp: float, ki: float, kd: float):
47         self.kp = kp; self.ki = ki; self.kd = kd
48         self.prev_error = 0.0
49         self.integral = 0.0
50         self.last_time = 0.0
51
52     def compute(self, error: float, sim_time: float) -> float:
53         dt = sim_time - self.last_time
54         if dt <= 0:
55             dt = 0.05
56
57         p_term = self.kp * error
58         self.integral += error * dt
59         self.integral = max(min(self.integral, 0.4), -0.4)
60         i_term = self.ki * self.integral
61         d_term = self.kd * (error - self.prev_error) / dt
62
63         self.prev_error = error
64         self.last_time = sim_time
65         return p_term + i_term + d_term
66
67     def reset(self, sim_time: float):
68         self.prev_error = 0.0; self.integral = 0.0; self.last_time = sim_time
69
70 pid_feedback = PIDController(Config.kp, Config.ki, Config.kd)
71
72 def get_ai_llm_decision(state: dict) -> str:
73     config = Config()
74     prompt = f"""
75 [SYSTEM]
76 You are the AI brain of a mobile robot Pioneer3DX. Analyze the state and select the best
77 macro action.
78 Available actions: "MOVE_FORWARD", "FOLLOW_WALL_RIGHT", "TURN_LEFT_90", "STOP_REACHED_GOAL".
79 [STATE DATA]
80 - Position: X={state['pos_x']:.2f}, Y={state['pos_y']:.2f}
81 - Goal: X={state['goal_x']:.2f}, Y={state['goal_y']:.2f}
82 - Distance to Goal: {state['dist_to_goal']:.2f} m
83 - Sonar Front Center: {state['sensors']['front_center']:.2f} m
84 - Sonar Front Right: {state['sensors']['front_right']:.2f} m
85 [RULES]
86 1. If Distance to Goal < 0.35, you MUST output "STOP_REACHED_GOAL".
87 2. If Sonar Front Center < {config.front_turn_dist}, you MUST output "TURN_LEFT_90".
88 3. If Sonar Front Right < 0.60, output "FOLLOW_WALL_RIGHT".
89 4. Else, output "MOVE_FORWARD".
90 Respond ONLY in valid JSON format: {"action": "<chosen_action>"}
91 """
92     try:
93         payload = {"model": config.model_name, "prompt": prompt, "stream": False, "format": "json"}
94         response = requests.post(config.ollama_url, json=payload, timeout=1.2)
95         return json.loads(response.json()['response']).strip().get("action", "MOVE_FORWARD")
96     except:
97         if state['sensors']['front_center'] < config.front_turn_dist: return "TURN_LEFT_90"
98         elif state['sensors']['front_right'] < 0.60: return "FOLLOW_WALL_RIGHT"
99         return "MOVE_FORWARD"
100 def coppelia_ai_interface(state: dict, action: str, sim_time: float) -> tuple:

```

```

101 config = Config()
102 sensors = state['sensors']
103
104 if action == "STOP_REACHED_GOAL":
105     return 0.0, 0.0
106 elif action == "TURN_LEFT_90":
107     return -config.base_speed * 0.4, config.base_speed * 0.4
108 elif action == "FOLLOW_WALL_RIGHT":
109     error = sensors['front_right'] - config.wall_dist
110     turn = pid_feedback.compute(error, sim_time) * -1.0
111     avoid_error = max(0.0, (config.wall_dist * 1.2) - sensors['front_left'])
112     avoid_turn = avoid_error * config.avoid_gain * -1.0
113
114     total_turn = turn + avoid_turn
115     max_turn_effect = config.max_turn * config.base_speed
116     total_turn = max(-max_turn_effect, min(max_turn_effect, total_turn))
117
118     left_joint_speed = config.base_speed - total_turn
119     right_joint_speed = config.base_speed + total_turn
120     return left_joint_speed, right_joint_speed
121 return config.base_speed, config.base_speed
122
123 def main():
124     coppelia = coppelia_client.Coppelia()
125     robot = coppelia_client.P3DX(coppelia.sim, "PioneerP3DX")
126     config = Config()
127     robot_handle = coppelia.sim.getObject("/PioneerP3DX")
128
129     running = True
130     def on_ctrl_c(sig, frame):
131         nonlocal running; running = False
132     signal.signal(signal.SIGINT, on_ctrl_c)
133
134     coppelia.start_simulation()
135     start_sim_time = coppelia.sim.getSimulationTime()
136     pid_feedback.reset(start_sim_time)
137     loop_counter = 0
138     ai_macro_action = "MOVE_FORWARD"
139
140     while running and coppelia.is_running():
141         current_sim_time = coppelia.sim.getSimulationTime()
142         pos_3d = coppelia.sim.getObjectPosition(robot_handle, -1)
143         pos_x, pos_y = pos_3d[0], pos_3d[1]
144         dist_to_goal = ((GOAL_POSITION[0] - pos_x)**2 + (GOAL_POSITION[1] - pos_y)**2)**0.5
145         vel_vector = coppelia.sim.getObjectVelocity(robot_handle)
146         actual_speed = (vel_vector[0][0]**2 + vel_vector[0][1]**2)**0.5
147         sonar_readings = SensorProcessor.process(robot.get_sonar())
148
149         current_state = {
150             "pos_x": pos_x, "pos_y": pos_y,
151             "goal_x": GOAL_POSITION[0], "goal_y": GOAL_POSITION[1],
152             "dist_to_goal": dist_to_goal,
153             "actual_speed": actual_speed,
154             "sensors": asdict(sonar_readings)
155         }
156
157         if loop_counter % 6 == 0:
158             ai_macro_action = get_ai_llm_decision(current_state)
159
160         left_wheel, right_wheel = coppelia_ai_interface(state=current_state, action=
ai_macro_action, sim_time=current_sim_time)
161         robot.set_speed(left_wheel, right_wheel)
162
163         if ai_macro_action == "STOP_REACHED_GOAL":
164             running = False
165             loop_counter += 1
166             time.sleep(0.04)
167
168         robot.set_speed(0, 0)
169         coppelia.stop_simulation()
170
171 if __name__ == "__main__":
172     main()

```

Listing 1: Script Implementasi State-Action Interface Pioneer P3DX

5 ANALISIS DAN PEMBAHASAN

Berdasarkan hasil uji coba jalannya program pada lingkungan labirin CoppeliaSim, ditemukan beberapa karakteristik performa sistem hibrida ini. Visualisasi lintasan pergerakan robot selama proses navigasi otonom dapat dilihat pada Gambar 1.

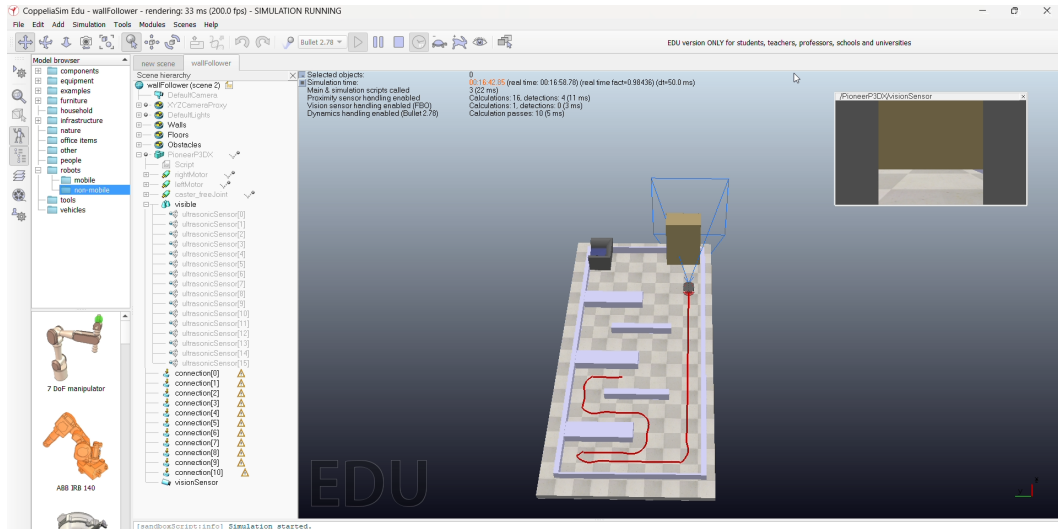


Figure 1: Uji Coba Lintasan dan Navigasi Robot Mobile Pioneer P3DX di Lingkungan Labirin Berbasis Prosedur Kendali Hibrida Berbantuan Simulator CoppeliaSim.

1. **Reduksi Efek Latency Pemrosesan AI:** Proses inferensi teks pada LLM lokal memerlukan waktu pengolahan (*computation time*) berkisar antara 50 ms hingga 150 ms. Jika pemanggilan API dilakukan pada setiap iterasi loop, pergerakan fisik robot akan menjadi tersendat (*stuttering*). Solusi penjadwalan pemanggilan berkala (setiap 6 siklus loop) terbukti efektif menjaga kelancaran dinamika gerak roda robot tanpa kehilangan kesadaran situasi lingkungan oleh AI.
2. **Efektivitas Mekanisme Fallback:** Keberadaan blok pengecualian (*try-except*) yang memicu fungsi aturan deterministik lokal (*fallback rule system*) menjamin robot tidak mengalami kecelakaan (*crash*) menabrak dinding ketika terjadi kegagalan transmisi data atau lonjakan beban kerja pada VRAM komputer.
3. **Analisis Kendali PID:** Implementasi pembatas nilai integral (*integral windup capping*) terbukti mampu mereduksi efek osilasi berlebih ketika robot berada di jalur lurus yang panjang, sehingga pergerakan robot saat menyusuri dinding kanan menjadi jauh lebih stabil dibandingkan sistem PID tanpa pengaman batas integral.

6 KESIMPULAN

Sistem navigasi hibrida yang menggabungkan kecerdasan semantik LLM lokal (Qwen2.5:1.5b) dan ketajaman fisis kontroler PID *closed-loop* telah berhasil dirancang dan diuji pada robot Pioneer3DX di simulator CoppeliaSim. Melalui arsitektur fungsi interface yang memisahkan pengolahan data *state* lingkungan dan eksekusi nilai *joint space*, robot mampu mengambil keputusan makro secara adaptif sekaligus melakukan manuver mikro secara presisi guna menyelesaikan misi navigasi menuju koordinat target secara aman dan otonom.